Introduction aux Bases de Données Relationnelles et Requêtes SQL de Base

Thème principal : Bases de données relationnelles, Modèle relationnel, Intégrité référentielle et Langage SQL ANSI

Synthèse exécutive : Ce cours magistral de la ressource R1.05, dispensé en BUT 1 Informatique à l'IUT de Bayonne et du Pays Basque, pose les fondations indispensables des bases de données relationnelles. Il détaille la transition critique des tableurs plats (Excel) vers les SGBD afin d'éliminer la redondance et les anomalies de mise à jour. Le cours formalise la structuration des données via les clés primaires (PK) et clés étrangères (FK) pour garantir l'intégrité référentielle, et enseigne l'écriture de requêtes SQL d'extraction normalisées selon la norme ISO/ANSI en utilisant des jointures explicites INNER JOIN.

1. Le Pourquoi : Des Tableurs aux Bases de Données Relationnelles

Dans les applications informatiques réelles, stocker des données dans un simple fichier plat ou un tableur comme Excel montre très vite ses limites structurelles. Si une information (telle que l'adresse postale d'un client) est répétée sur 50 lignes de commandes différentes, le moindre changement d'adresse impose d'effectuer 50 modifications manuelles. Si une seule ligne est omise, la base souffre d'une anomalie de mise à jour et devient incohérente. Le modèle relationnel résout ce problème fondamental en séparant les entités distinctes (par exemple, la table 'Clients' d'un côté et la table 'Commandes' de l'autre) et en établissant des liaisons logiques rigoureuses.

- Problématique des tableurs plats : Redondance de données et risques majeurs d'anomalies de mise à jour.
- SGBD (Système de Gestion de Bases de Données) : Logiciel assurant la cohérence, la sécurité, l'accès concurrentiel et la persistance.
- Modèle Relationnel : Découpage des informations en entités distinctes (tables) reliées logiquement pour éviter la duplication.

Cas concret enseignant : *L'enseignant a illustré cette problématique avec l'analogie du système d'information bancaire : si une banque enregistrait le nom, le prénom et l'adresse du titulaire sur chaque ligne de virement ou de paiement par carte, un simple déménagement exigerait la mise à jour de milliers d'historiques passés, générant un risque critique d'erreur de saisie et une saturation inutile de l'espace de stockage.*

2. Clés Primaires, Clés Étrangères et Intégrité Référentielle

Pour qu'une base de données soit exploitable sans confusion, chaque enregistrement (n-uplet) doit pouvoir être identifié de manière unique. La Clé Primaire (PK) remplit ce rôle critique : elle garantit l'unicité et refuse les valeurs nulles. Lorsque nous souhaitons associer deux entités entre elles (par exemple rattacher un étudiant à son groupe de TD), nous n'dupliquons pas toutes les données du groupe dans la table Étudiant : nous y insérons simplement l'identifiant du groupe sous forme de Clé Étrangère (FK). Le SGBD vérifie alors en permanence la contrainte d'intégrité référentielle.

- Clé Primaire (Primary Key - PK) : Attribut ou groupe d'attributs identifiant de façon unique et non ambiguë chaque ligne (n-uplet) d'une table, obligatoire et strictement non nul (NOT NULL).
- Clé Étrangère (Foreign Key - FK) : Attribut présent dans une table qui fait référence directe à la clé primaire d'une autre table pour matérialiser une association.
- Contrainte d'Intégrité Référentielle : Règle métier automatisée par le SGBD qui interdit d'insérer ou de maintenir une clé étrangère pointant vers un identifiant inexistant.

Cas concret enseignant : *L'enseignant a pris l'exemple du Numéro de Sécurité Sociale (NIR) : même si deux étudiants portent exactement les mêmes nom et prénom au Pays Basque, leur clé primaire unique (NIR ou id_etudiant) empêche tout risque de confusion ou de doublon au sein du système d'information de l'IUT.*

3. Extraction de Données en SQL ANSI : SELECT, WHERE et INNER JOIN

Le Structured Query Language (SQL) est le langage standard d'interrogation des bases de données relationnelles. Pour croiser des informations réparties dans plusieurs tables, la norme moderne ISO/ANSI impose d'utiliser des jointures explicites. La structure fondamentale d'une requête multi-table s'écrit : SELECT colonnes FROM table1 INNER JOIN table2 ON table1.fk = table2.pk WHERE conditions. L'utilisation d'alias de tables (ex: 'Etudiant e') est une bonne pratique professionnelle pour clarifier l'écriture et éviter toute ambiguïté sur les noms de colonnes.

- Langage Déclaratif : Le SQL exprime CE QUE l'on souhaite obtenir et non les étapes algorithmiques pour y parvenir.
- Projection (SELECT) : Choix explicite des colonnes à extraire et afficher dans le jeu de résultats.
- Filtrage (WHERE) : Application de critères logiques pour cibler uniquement les lignes répondant à des conditions précises.
- Jointure Explicite (INNER JOIN ... ON) : Combinaison de deux tables basée sur l'égalité stricte entre la clé étrangère et la clé primaire.

Cas concret enseignant : *Mise en garde stricte du professeur pour les partiels : 'N'utilisez jamais la syntaxe obsolète du produit cartésien avec une virgule dans le FROM (ex: FROM Etudiant, Groupe WHERE Etudiant.id_g = Groupe.id). Utilisez TOUJOURS la syntaxe explicite INNER JOIN ... ON. Le produit cartésien implicite fait perdre des points en examen et est formellement banni en entreprise.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne doit-on pas utiliser SELECT * dans un code applicatif ou lors d'un examen ?
Réponse du professeur : Parce que SELECT * récupère l'intégralité des colonnes d'une table sans discernement. En entreprise, cela surcharge le réseau inutilement si la table contient des champs lourds (comme des blobs ou des commentaires). En examen, cela montre une absence de rigueur dans la projection et empêche de repérer si l'étudiant a compris quelles colonnes exactes étaient demandées.

Question étudiant : Que se passe-t-il si j'essaie de supprimer un groupe dans lequel il y a encore des étudiants inscrits ?
Réponse du professeur : Le SGBD refuse catégoriquement l'opération et déclenche une erreur d'intégrité référentielle. C'est précisément le rôle de la contrainte de clé étrangère : empêcher qu'un étudiant se retrouve rattaché à un groupe fantôme qui n'existe plus.

■ Exercice & Démonstration d'Application

Énoncé : On considère le schéma relationnel suivant : - Etudiant (id_etudiant, nom, prenom, #id_groupe) - Groupe (id_groupe, nom_groupe, promo) Écrire la requête SQL conforme à la norme ISO/ANSI permettant d'extraire le nom et le prénom des étudiants appartenant à la promotion 'BUT1-INFO' et inscrits dans le groupe nommé 'A1'.

Résolution pas à pas :
Étape 1 : Identifier les tables nécessaires et leur attribuer des alias clairs -> Etudiant (e) et Groupe (g). Étape 2 : Établir la jointure explicite en reliant la clé étrangère de la table Etudiant à la clé primaire de la table Groupe -> INNER JOIN Groupe g ON e.id_groupe = g.id_groupe. Étape 3 : Poser les conditions de filtrage dans la clause WHERE -> g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1'. Étape 4 : Définir la projection exacte des attributs demandés -> SELECT e.nom, e.prenom.

Résultat : SELECT e.nom, e.prenom FROM Etudiant e INNER JOIN Groupe g ON e.id_groupe = g.id_groupe WHERE g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1';

■ Auto-Évaluation Flash d'Examen

Q : Quelles sont les deux propriétés fondamentales d'une Clé Primaire (Primary Key) ?

R : Elle doit garantir l'unicité de chaque enregistrement dans la table et être strictement non nulle (NOT NULL).

Q : À quoi sert la clause INNER JOIN en SQL et sur quoi doit porter sa condition ON ?

R : Elle sert à combiner et croiser explicitement les lignes de deux tables en faisant correspondre la clé étrangère de l'une avec la clé primaire de l'autre.

Q : Pourquoi la syntaxe 'FROM TableA, TableB WHERE TableA.id = TableB.id_a' est-elle pénalisée en partiel ?

R : Parce qu'il s'agit d'une syntaxe implicite obsolète (produit cartésien) non conforme aux bonnes pratiques de la norme ISO/ANSI exigée en enseignement supérieur et en entreprise.